MemoryProfiler2

一、简介

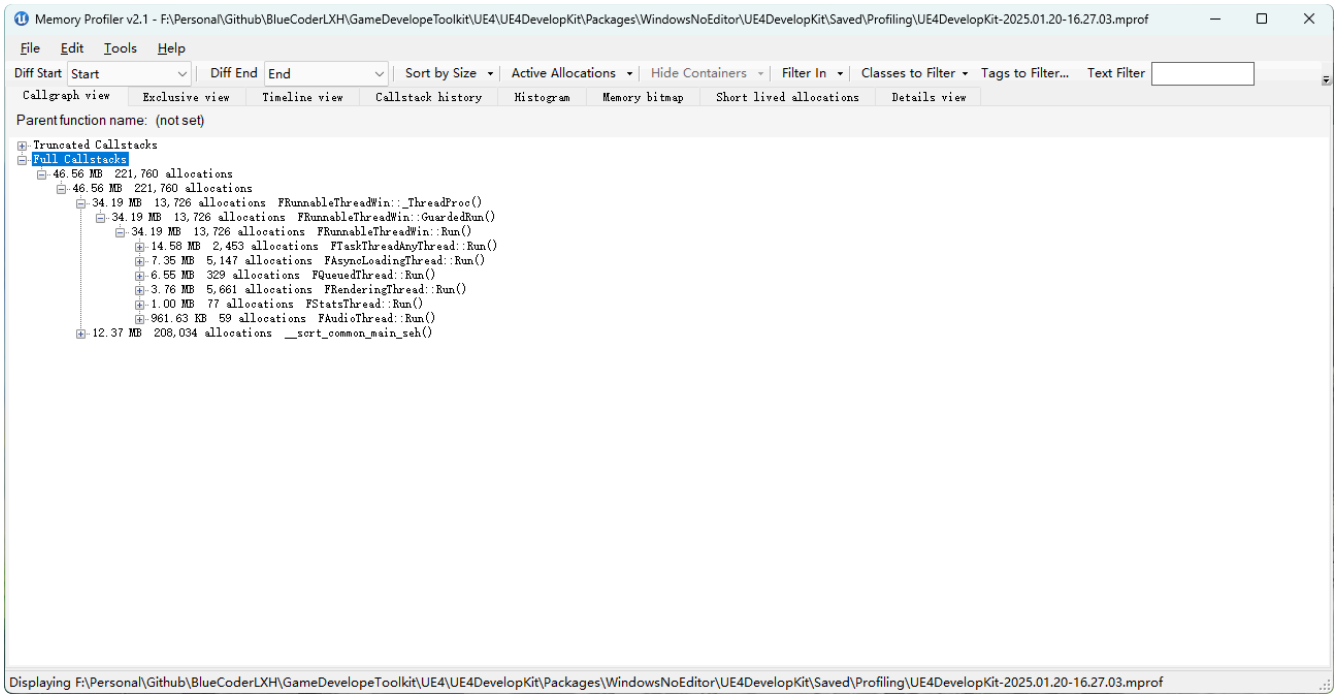
MemoryProfiler2 是 UE 提供的一个分析详细内存数据的工具，主要分两大块：

- **FMallocProfiler**

MemoryProfiler2 工具逻辑层面的实现，包括捕获内存快照、Memory Tag、StackBackTrace 以及生成 .mprof 文件等

- **Programs/MemoryProfiler2**

.mprof 文件的解析工具



二、基本原理以及优化

2.1 基本原理

下面简单讲一下 FMallocProfiler 生成内存快照的原理：

FMallocProfiler 本身继承于 FMalloc，并在内部内嵌游戏真正使用的 FMalloc

C/C++

```
class CORE_API FMallocProfiler : public FMalloc
{
    ...
protected:
    /** Malloc we're based on, aka using under the hood
                                    */
    FMalloc*                                UsedMalloc;
    ...
};
```

声明 FMallocProfiler 类型的全局指针 GMallocProfiler

C/C++

```
CORE_API FMallocProfiler* GMallocProfiler;
```

在创建 Malloc 时，覆盖原有的 Malloc

C/C++

```cpp
/** Helper function called on first allocation to create and initialize GMalloc */
static int FMemory_GCreateMalloc_ThreadUnsafe()
{
#if !PLATFORM_MAC
    FPlatformMemoryStats Stats = FPlatformMemory::GetStats();
    uint64 ProgramSize = Stats.UsedPhysical;
#endif

    GMalloc = FPlatformMemory::BaseAllocator();
    // Setup malloc crash as soon as possible.
    FPlatformMallocCrash::Get( GMalloc );
    ...
// so now check to see if we are using a Mem Profiler which wraps the GMalloc
#if USE_MALLOC_PROFILER
    #if WITH_ENGINE && IS_MONOLITHIC
        GMallocProfiler = new FMallocProfilerEx( GMalloc );
    #else
        GMallocProfiler = new FMallocProfiler( GMalloc );
    #endif

    // Don't do 'BeginProfiling' here for it's too early
    // GMallocProfiler->BeginProfiling();

    GMalloc = GMallocProfiler;
#endif

    ...
    return 0;
}
```

重载函数 Malloc、Realloc、Free，即在内存分配和释放时，捕获内存快照数据

C/C++

```
class CORE_API FMallocProfiler : public FMalloc
{
    ...
    /**
     * Malloc
     */
    virtual void* Malloc( SIZE_T Size, uint32 Alignment ) override
    {
        if (Size == 0)
        {
            return nullptr;
        }

        FScopeLock Lock( &CriticalSection );
        void* Ptr = UsedMalloc->Malloc( Size, Alignment );

        if (IsOutsideTrackingFunction())
        {
            FScopedMallocProfilerLock MallocProfilerLock(*this);
            TrackMalloc( Ptr, (uint32)Size );
            TrackSpecialMemory();
        }

        return Ptr;
    }

    /**
     * Realloc
     */
    virtual void* Realloc( void* OldPtr, SIZE_T NewSize, uint32 Alignment ) override
    {
        FScopeLock Lock( &CriticalSection );
        void* NewPtr = UsedMalloc->Realloc( OldPtr, NewSize, Alignment );

        if (IsOutsideTrackingFunction())
        {
```

```
            FScopedMallocProfilerLock MallocProfilerLock(*this);
            TrackRealloc( OldPtr, NewPtr, (uint32)NewSize );
            TrackSpecialMemory();
        }

        return NewPtr;
    }

    /**
     * Free
     */
    virtual void Free( void* Ptr ) override
    {
        if (Ptr == nullptr)
        {
            return;
        }
        FScopeLock Lock( &CriticalSection );
        UsedMalloc->Free( Ptr );

        if (IsOutsideTrackingFunction())
        {
            FScopedMallocProfilerLock MallocProfilerLock(*this);
            TrackFree( Ptr );
            TrackSpecialMemory();
        }
    }
    ...
};
```

2.2 工具优化

引擎默认会在打开 MemoryProfiler2 功能后就开始捕获内存快照，并且整个生命周期只允许执行一次，且记录数据通常较大，不适合记录太长时间。由于我们游戏有时候会超过 10 分钟，因此有必要对工具进行优化改造：

- **支持选择 BeginProfiling 执行时机**

目前默认两种：加载地图时或执行对应命令

- **支持添加自定义 marker**

- **支持多次捕获内存快照**

三、用法

3.1 命令

常用相关命令：

- **g.EnableMallocProfileOnLoadMap**

下一次加载地图时自动开始记录 memory snapshot

- **MPROF START**

手动开始记录

- **MPROF STOP**

手动停止记录（必须执行，否则 mprof 文件无法打开）

- **MPROF MARK**

添加自定义标签

添加标签除了用命令，还可以用代码：

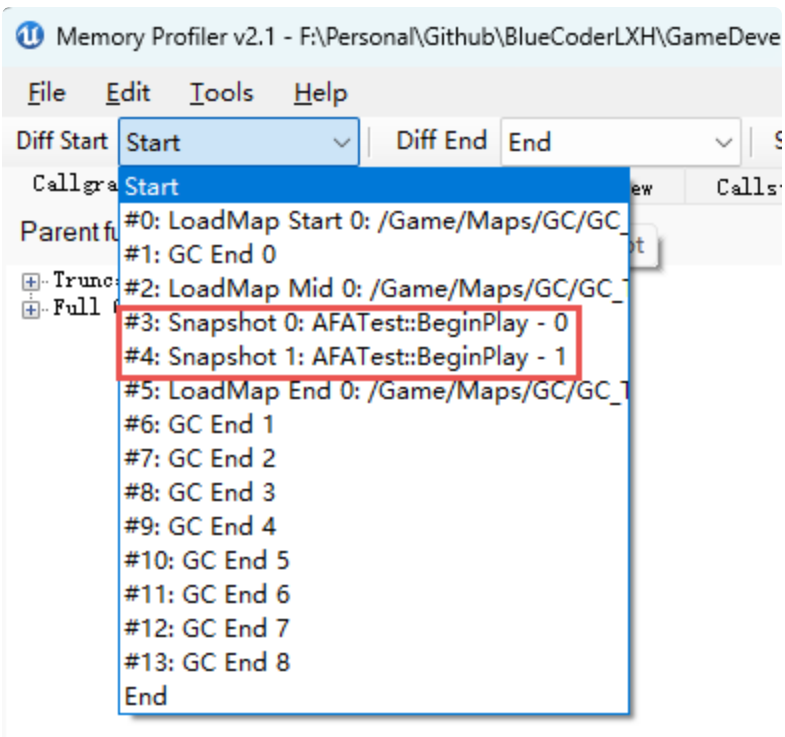
C/C++

```
MALLOC_PROFILER(FMallocProfiler::SnapshotMemoryCustom(TEXT("MarkerName")));
```

例如：

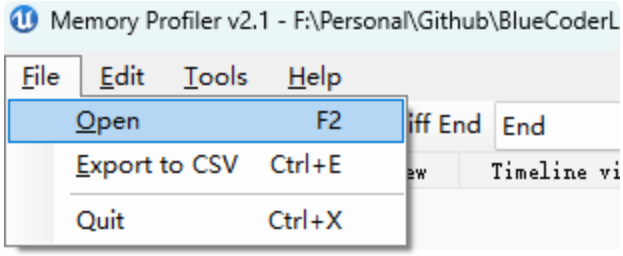
C/C++

```cpp
void AFATest::BeginPlay()
{
    MALLOC_PROFILER(FMallocProfiler::SnapshotMemoryCustom(TEXT("AFATest::BeginPlay - 0")));

    Super::BeginPlay();

    ...

    MALLOC_PROFILER(FMallocProfiler::SnapshotMemoryCustom(TEXT("AFATest::BeginPlay - 1")));
}
```



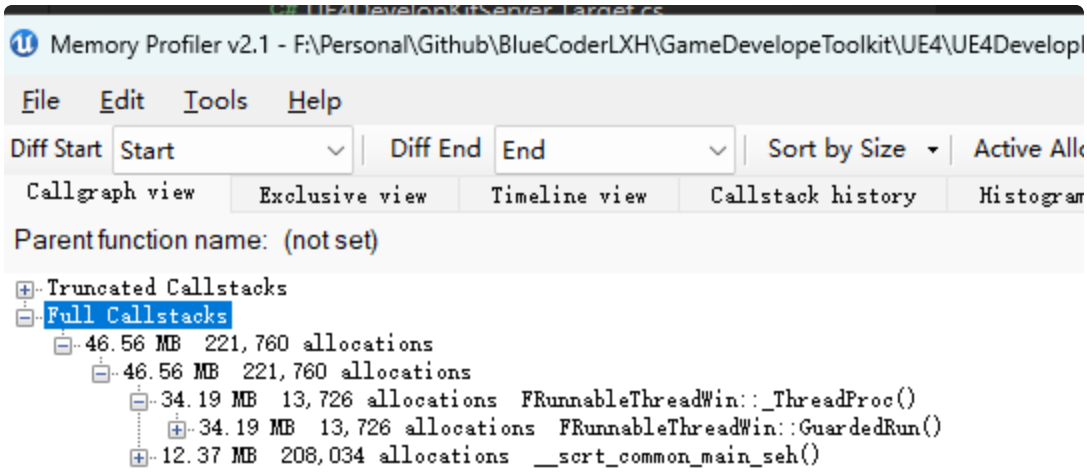
3.2 MemoryProfiler2

- **打开 mprof 文件**

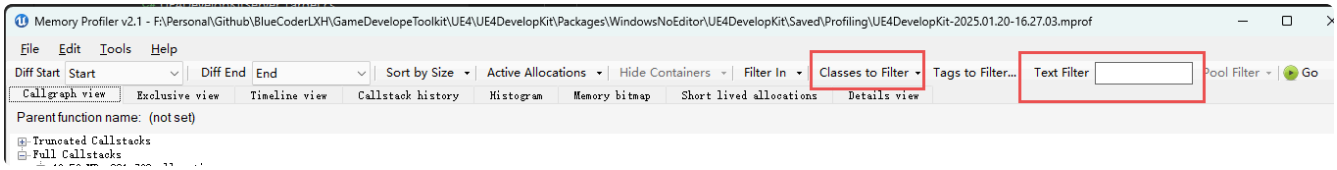

Memory Profiler v2.1 - F:\Personal\Github\BlueCoderL
File
Edit
Tools
Help
Open F2
Export to CSV Ctrl+E
Quit Ctrl+X
iff End
End
Timeline vi

Memory Profiler v2.1 - F:\Personal\Github\BlueCoderLXH\GameDevelopeToolkit\UE4\UE4DevelopKit\Packages\WindowsNoEditor\UE4DevelopKit\Saved\Profiling\UE4DevelopKit-2025.01.20-16.27.03.mprof
File
Edit
Tools
Help
Diff Start
Start
Diff End
End
Sort by Size
Active Allocations
Hide Containers
Filter In
Classes to Filter
Tags to Filter...
Text Filter
Pool Filter
Go
Callgraph view
Exclusive view
Timeline view
Callstack history
Histogram
Memory bitmap
Short lived allocations
Details view
Parent function name: (not set)
Parsed F:\Personal\Github\BlueCoderLXH\GameDevelopeToolkit\UE4\UE4DevelopKit\Packages\WindowsNoEditor\UE4DevelopKit\Saved\Profiling\UE4DevelopKit-2025.01.20-16.27.03.mprof

- **展开堆栈**

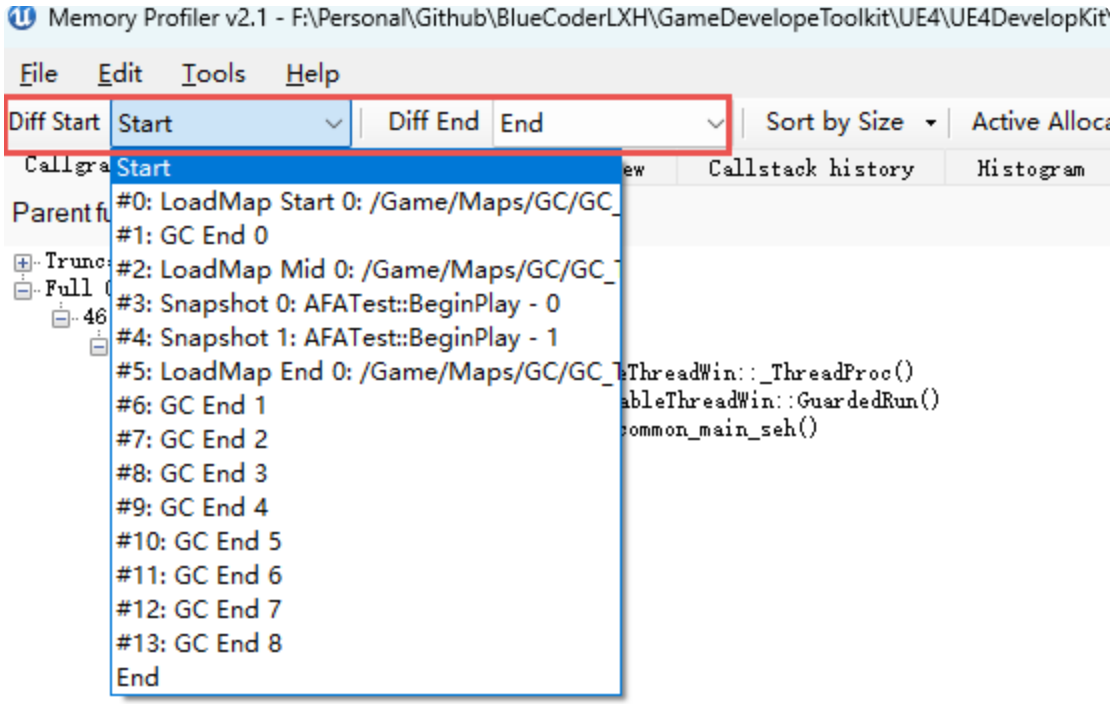

Memory Profiler v2.1 - F:\Personal\Github\BlueCoderLXH\GameDevelopeToolkit\UE4\UE4Develop
File
Edit
Tools
Help
Diff Start
Start
Diff End
End
Sort by Size
Active All
Callgraph view
Exclusive view
Timeline view
Callstack history
Histogram
Parent function name: (not set)
Truncated Callstacks
Full Callstacks
46.56 MB 221,760 allocations
46.56 MB 221,760 allocations
34.19 MB 13,726 allocations FRunnableThreadWin::_ThreadProc()
34.19 MB 13,726 allocations FRunnableThreadWin::GuardedRun()
12.37 MB 208,034 allocations __scrt_common_main_seh()

- **搜索**



可以直接搜索堆栈或过滤 Class

- **选择 marker**



marker 的作用是对需要　分析的场景分段，例如只需要战斗内某个玩法或者某个技能释放的内存开销

- **内存整体数据**

Size | Active Allocations | Hide Containers | Filter In | Classes to Filter | Tags to Filter... | Text Filter | Pool Filter

istory | Histogram | Memory bitmap | Short lived allocations | Details view

Diff

```
Description: End
Stream Index: 17440143
Frame Number: 2108
Current Time: 82.93002 seconds
Elapsed Time: 9.287463 seconds
Snapshot Type: SUBTYPE_SnapshotMarker
Snapshot Index: 0
Loaded Levels: 0
Memory Allocation Stats:
        Total Physical: 63.02 GB (67,666,944,000 bytes)
        Total Virtual: 128.00 TB (140,737,488,224,256 bytes)
        Page Size: 4.00 KB (4,096 bytes)
        Total Physical GB: 64.00 bytes (64 bytes)
        Available Physical: 32.25 GB (34,624,638,976 bytes)
        Available Virtual: 36.31 GB (38,986,698,752 bytes)
        Used Physical: 1.99 GB (2,133,655,552 bytes)
        Peak Used Physical: 1.99 GB (2,133,790,720 bytes)
        Used Virtual: 2.80 GB (3,005,792,256 bytes)
        Peak Used Virtual: 2.80 GB (3,005,960,192 bytes)
        Windows Specific Memory Stat: 0.00 bytes (0 bytes)
        Malloc calls: 0.00 bytes (0 bytes)
        Realloc calls: 0.00 bytes (0 bytes)
        Free calls: 0.00 bytes (0 bytes)
        Total Allocator calls: 0.00 bytes (0 bytes)
        Memory Profiling Overhead: 3.75 MB (3,927,432 bytes)
Allocation Count: 17,588,605
Allocation Size: 46.58 MB (48,846,734 bytes)
Allocation Max Size: 46.71 MB (48,977,780 bytes)
Is Diff Result: False
```

End


```
Description: End
Stream Index: 17440143
Frame Number: 2108
Current Time: 82.93002 seconds
Elapsed Time: 9.287463 seconds
Snapshot Type: SUBTYPE_SnapshotMarker
Snapshot Index: 0
Loaded Levels: 0
Memory Allocation Stats:
        Total Physical: 63.02 GB (67,666,944,000 bytes)
        Total Virtual: 128.00 TB (140,737,488,224,256 bytes)
        Page Size: 4.00 KB (4,096 bytes)
        Total Physical GB: 64.00 bytes (64 bytes)
        Available Physical: 32.25 GB (34,624,638,976 bytes)
        Available Virtual: 36.31 GB (38,986,698,752 bytes)
        Used Physical: 1.99 GB (2,133,655,552 bytes)
        Peak Used Physical: 1.99 GB (2,133,790,720 bytes)
        Used Virtual: 2.80 GB (3,005,792,256 bytes)
        Peak Used Virtual: 2.80 GB (3,005,960,192 bytes)
        Windows Specific Memory Stat: 0.00 bytes (0 bytes)
        Malloc calls: 0.00 bytes (0 bytes)
        Realloc calls: 0.00 bytes (0 bytes)
        Free calls: 0.00 bytes (0 bytes)
        Total Allocator calls: 0.00 bytes (0 bytes)
        Memory Profiling Overhead: 3.75 MB (3,927,432 bytes)
Allocation Count: 17,588,605
Allocation Size: 46.58 MB (48,846,734 bytes)
Allocation Max Size: 46.71 MB (48,977,780 bytes)
Is Diff Result: False
```